PRACTICALS INDEX

All 15 supplied practicals — photo question excluded

No.	Practical
1	Form Validation using JavaScript
2	Static Web Pages – Online Book Store
3	Angular Application – Arithmetic Operations
4	AngularJS Shopping List
5	Simple Calculator using React JS
6	Voting Application using React JS
7	Accessing MongoDB from Node.js
8	Manipulating MongoDB from Node.js
9	Fetch API – Display Data
10	NodeJS Static Server without Express
11	NodeJS Express Server – JSON / Handlebars
12	NodeJS Server with MongoDB
13	NodeJS Server with MySQL
14	Simple Login Form using React JS
15	Chat Module using HTML, CSS and JavaScript

Practical 1

Form Validation using JavaScript

STEPS TO PERFORM

1. Open VS Code (or Notepad) and create a new folder, for example `form_validation`.
2. Inside the folder create a new file named `form_validation.html`.
3. Type or paste the program: the `<script>` holds the `VALIDATEDETAIL()` function and the `<form name="RegForm">` holds the input fields.
4. Save the file with `Ctrl + S`.
5. Open the file in a browser (double-click it, or right-click and choose Open with Live Server).
6. Test 1: click Send with empty fields. An alert asks for the first missing field.
7. Test 2: type an e-mail without `@` or `.`. An alert asks for a valid e-mail address.
8. Test 3: fill every field correctly and click Send. The form submits with no alert.
9. Take a screenshot of the output for your record.

PROGRAM / DETAILS

AIM: To create Form validation using JavaScript.

Procedure:

1. Project Setup

• Create HTML Structure:

– Create an HTML file (e.g., form_validation.html) with the following:

· A form element (<form>) with an appropriate id (e.g., myForm).

· Input fields for each data point (e.g., name, email, phone).

· A submit button (<button type="submit">).

2. JavaScript Implementation

• Create a JavaScript file (e.g., script.js) and link it to your HTML file.

• Define Validation Functions:

– Create separate functions for each validation rule (e.g., validateName(), validateEmail(), validatePhone()).

– Implement the logic for each validation rule within the respective functions.

· Example:

```
| validateName(): Check if the name field is not empty and
contains only letters and spaces.
| validateEmail(): Check if the email field follows the standard
email format (e.g., using a regular expression).
| validatePhone(): Check if the phone number field contains only
digits and is of the correct length.
```

• Event Handling:

– Attach an event listener to the form's submit event.

– Inside the event handler:

· Call all the validation functions for each input field.

· If any validation fails:

· Display an appropriate error message (e.g., within the span element).

· Prevent the form from submitting (using event.preventDefault()).

· If all validations pass:

· Submit the form (allow the default form submission behavior).

3. Test the Application Program

```
<html>
<head>
<script>
function VALIDATEDETAIL()
{
var name = document.forms["RegForm"]["Name"];
var email = document.forms["RegForm"]["EMail"];
var phone = document.forms["RegForm"]["Telephone"];
var what = document.forms["RegForm"]["Subject"];
var password = document.forms["RegForm"]["Password"];
var address = document.forms["RegForm"]["Address"];
if (name.value == "")
{
window.alert("Please enter your name.");
name.focus();
return false;
}
if (address.value == "")
{
window.alert("Please enter your address.");
name.focus();
return false;
}
if (email.value == "")
{
```

```
window.alert("Please enter a valid e
-mail address.");
email.focus();
return false;
}
if (email.value.indexOf("@", 0) < 0)
{
window.alert("Please enter a valid e
-mail address.");
email.focus();
return false;
}
if (email.value.indexOf(".", 0) < 0)
{
window.alert("Please enter a valid e
-mail address.");
email.focus();
return false;
}
if (phone.value == "")
{
window.alert("Please enter your telephone number.");
phone.focus();
return false;
}
if (password.value == "")
{
window.alert("Please enter your password");
password.focus();
return flase;
}
if (what.selectedIndex < 1)
{
alert("Please enter your course.");
what.focus();
return false;
}
return true;
}</script>
<style>
VALIDATEDETAIL {
font-weight: bold ;
float: left;
width: 100px;
text-align: left;
margin-right: 10px;
font-size:14px;
}
div {
box-sizing: border-box;
width: 100%;
border: 100px solid black;
```

```
float: left;
align-content: center;
align-items: center;
}
form {
margin: 0 auto;
width: 600px;
}</style></head>
<body>
<h1 style="text-align: center"> REGISTRATION FORM </h1>
<form name="RegForm" action="submit.php" onsubmit="return
    VALIDATEDETAIL()"
method="post">
<p>Name: <input type="text" size=65 name="Name"> </p><br>
<p> Address: <input type="text" size=65 name="Address"> </p><br>
<p>E-mail Address: <input type="text" size=65 name="EMail"> </p><br>
<p>Password: <input type="text" size=65 name="Password"> </p><br>
<p>Telephone: <input type="text" size=65 name="Telephone"> </p><b
>SELECT YOUR COURSE
<select type="text" value="" name="Subject">
<option>BTECH</option>
<option>BBA</option>
<option>BCA</option>
<option>B.COM</option>
<option>VALIDATEDETAIL</option>
</select></p><br><br>
<p>Comments: <textarea cols="55" name="Comment"> </textarea></p>
<p><input type="submit" value="send" name="Submit">
<input type="reset" value="Reset" name="Reset
</p>
</form>
</body>
</html>
```

Practical 2

Static Web Pages - Online Book Store

STEPS TO PERFORM

1. Create a folder `bookstore` and a file `index.html` inside it.
2. Paste the HTML, the CSS inside `<style>` and the JavaScript inside `<script>` as given below.
3. Save the file and make sure the internet is on (the book cover images load from a web link).
4. Open it in a browser or with Live Server.
5. Click Home, Catalog and Contact in the menu to jump to each section.
6. Hover over a Buy Now button to see the colour change.
7. Fill the Contact form and click Send Message. A thank-you alert appears.
8. Take a screenshot of the page.

PROGRAM / DETAILS

Designing following static WebPages required for an Online Book Store website

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Online Bookstore</title>
<style>
* {
margin: 0;
padding: 0;
box-sizing: border-box;
}
body {
font-family: Arial, sans-serif;
line-height: 1.6;
}
header {
background-color: #333;
color: white;
padding: 10px 0;
text-align: center;
}
header .logo h1 {
margin: 0;
```

```
}
nav ul {
list-style-type: none;
padding: 0;
}
nav ul li {
display: inline;
margin: 0 15px;
}
nav ul li a {
color: white;
text-decoration: none;
font-size: 1.1em;
}
nav ul li a:hover {
color: #f0a500;
}
section.hero {
background-color: #f4f4f4;
padding: 20px;
text-align: center;
}
section.catalog {
display: flex;
justify-content: space-around;
flex-wrap: wrap;
padding: 20px;
}
section.catalog .book {
width: 30%;
margin-bottom: 20px;
text-align: center;
}
section.catalog .book img {
width: 100%;
height: auto;
}
section.catalog .book h3 {
font-size: 1.2em;
margin: 10px 0;
}
section.catalog .book p {
font-size: 1em;
color: #555;
}
section.catalog .book a {
background-color: #333;
color: white;
padding: 10px;
text-decoration: none;
display: inline-block;
margin-top: 10px;
```

```
}
section.catalog .book a:hover {
background-color: #f0a500;
}
section.contact {
padding: 20px;
}
section.contact form {
max-width: 600px;
margin: 0 auto;
}
section.contact form input,
section.contact form textarea {
width: 100%;
padding: 10px;
margin-bottom: 10px;
border: 1px solid #ccc;
border-radius: 4px;
}
section.contact form button {
background-color: #333;
color: white;
padding: 10px 20px;
border: none;
cursor: pointer;
}
section.contact form button:hover {
background-color: #f0a500;
}
footer {
background-color: #333;
color: white;
text-align: center;
padding: 10px;
margin-top: 20px;
}
</style>
</head>
<body>
<header>
<div class="logo">
<h1>Online Bookstore</h1>
</div>
<nav>
<ul>
<li><a href="#home">Home</a></li>
<li><a href="#catalog">Catalog</a></li>
<li><a href="#contact">Contact</a></li>
</ul>
</nav>
</header>
<!-- Hero Section -->
```

```
<section id="home" class="hero">
<h2>Welcome to the Best Bookstore!</h2>
<p>Browse and shop from a wide variety of books across all genres.</p>
</section>
<!-- Book Catalog -->
<section id="catalog" class="catalog">
<h2>Our Book Collection</h2>
<div class="book">
<img src="https://via.placeholder.com/150" alt="Book Cover">
<h3>Book Title 1</h3>
<p>Description of Book 1.</p>
<a href="#">Buy Now</a>
</div>
<div class="book">
<img src="https://via.placeholder.com/150" alt="Book Cover">
<h3>Book Title 2</h3>
<p>Description of Book 2.</p>
<a href="#">Buy Now</a>
</div>
<div class="book">
<img src="https://via.placeholder.com/150" alt="Book Cover">
<h3>Book Title 3</h3>
<p>Description of Book 3.</p>
<a href="#">Buy Now</a>
</div>
</section>
<!-- Contact Section -->
<section id="contact" class="contact">
<h2>Contact Us</h2>
<p>If you have any questions, feel free to reach out!</p>
<form>
<label for="name">Your Name:</label>
<input type="text" id="name" name="name" required>
<label for="email">Your Email:</label>
<input type="email" id="email" name="email" required>
<label for="message">Your Message:</label>
<textarea id="message" name="message" rows="4" required></textarea>
<button type="submit">Send Message</button>
</form>
</section>
<footer>
<p>&copy; 2024 Online Bookstore. All rights reserved.</p>
</footer>
<!-- JavaScript -->
<script>
document.querySelector('form').addEventListener('submit', function(e) {
e.preventDefault();
alert('Thank you for contacting us! We will get back to you soon.');
});
</script>
</body>
</html>
```

Practical 3

Angular Application - Arithmetic Operations

STEPS TO PERFORM

1. Create a file `arithmetic.html` in a new folder.
2. Paste the code below. It loads AngularJS from a CDN link, so the internet must be on.
3. Save the file and open it in a browser.
4. Type two numbers in the Add boxes. The sum appears instantly next to the `=` sign.
5. Repeat for Subtract, Multiply and Divide.
6. Change any number and watch the answer update live (two-way binding with `ng-model`).
7. Take a screenshot of the output.

PROGRAM / DETAILS

Aim: Create an Angular application which add, multiply, divide the two input numbers from textbox.

```
<!Doctype html>
<html lang="en">
<head>
<meta charset="utf-8">
<title>AngularApp</title>
<base href="/">
<meta name="viewport" content="width=device-width, initial-scale=1">
<link rel="icon" type="image/x-icon" href="favicon.ico">
<script
    src="https://cdnjs.cloudflare.com/ajax/libs/angular.js/1.8.3/angular
    .js"></script>
</head>
<body ng-app>
<h1>First AngularJS Application</h1>
```

Enter Numbers to Add:

```
<input type="number" ng-model="Num1" /> + <input type="number" ngmodel="
Num2" /> = <span>{{Num1 + Num2}}</span> <br><br>
```

Enter Numbers to Subtract:

```
<input type="number" ng-model="Num3" /> - <input type="number" ngmodel="
Num4" /> = <span>{{Num3 - Num4}}</span> <br><br>
```

Enter Numbers to Multiply:

```
<input type="number" ng-model="Num5" /> x <input type="number" ngmodel="
Num6" /> = <span>{{Num5 * Num6}}</span> <br><br>
```

Enter Numbers to Divide:

```
<input type="number" ng-model="Num7" /> / <input type="number" ngmodel="
Num8" /> = <span>{{Num7 / Num8}}</span>
</body>
</html>
```

Practical 4

AngularJS Shopping List

STEPS TO PERFORM

1. Create a folder `shopping-list` and a file `index.html`.
2. Paste the code below. `ng-app` and `ng-controller` connect the page to the AngularJS module and controller.
3. Save and open the file in a browser (internet needed for the AngularJS CDN).
4. Type an item name in the text box and click Add Item. It appears in the list (`ng-repeat`).
5. Click Remove next to an item to delete it (`ng-click` with `removeItem($index)`).
6. Remove all items to see the message No items in the list.
7. Take a screenshot of the output.

PROGRAM / DETAILS

Develop an Angular JS application that displays a list of shopping items. Allow users to add and remove items from the list using directives and controllers. Note: The default values of items may be included in the program

To develop an AngularJS application that displays a list of shopping items and allows users to add and remove items, we'll need to create a simple AngularJS app with the following features:

1. Display a list of items.
2. Allow users to add new items to the list.
3. Allow users to remove items from the list.
4. Use AngularJS directives (ng-repeat, ng-click) and controllers.

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Shopping List App</title>
<script
    src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.m
    in.js"></script>
<style>
```

```
body {
font-family: Arial, sans-serif;
padding: 20px;
background-color: #f9f9f9;
}
h2 {
text-align: center;
color: #4CAF50;
}
.shopping-list {
max-width: 500px;
margin: 0 auto;
background-color: #fff;
padding: 20px;
border-radius: 8px;
box-shadow: 0 4px 8px rgba(0, 0, 0, 0.1);
}
.shopping-list input[type="text"] {
width: 80%;
padding: 8px;
font-size: 16px;
border-radius: 4px;
border: 1px solid #ccc;
margin-right: 10px;
}
.shopping-list button {
padding: 8px 16px;
font-size: 16px;
border: none;
border-radius: 4px;
background-color: #4CAF50;
color: white;
cursor: pointer;
}
.shopping-list button:hover {
background-color: #45a049;
}
.item {
display: flex;
justify-content: space-between;
align-items: center;
margin: 10px 0;
}
.item button {
background-color: #f44336;
border: none;
color: white;
font-size: 16px;
cursor: pointer;
}
.item button:hover {
background-color: #e53935;
```

```
}
</style>
</head>
<body ng-app="shoppingApp" ng-controller="shoppingCtrl">
<h2>Shopping List</h2>
<div class="shopping-list">
<input type="text" ng-model="newItem" placeholder="Enter item" />
<button ng-click="addItem()">Add Item</button>
<div ng-if="shoppingList.length > 0">
<h3>Items:</h3>
<ul>
<li ng-repeat="item in shoppingList">
<div class="item">
<span>{{ item }}</span>
<button ng-click="removeItem($index)">Remove</button>
</div>
</li>
</ul>
</div>
<div ng-if="shoppingList.length === 0">
<p>No items in the list.</p>
</div>
</div>
<script>
// AngularJS Application
var app = angular.module("shoppingApp", []);
// Controller for the shopping list
app.controller("shoppingCtrl", function($scope) {
// Initialize the shopping list
$scope.shoppingList = [];
// Add item to the shopping list
$scope.addItem = function() {
if ($scope.newItem && $scope.newItem.trim() !== "") {
$scope.shoppingList.push($scope.newItem.trim());
$scope.newItem = ""; // Clear the input field
}
};
// Remove item from the shopping list
$scope.removeItem = function(index) {
$scope.shoppingList.splice(index, 1);
};
});
</script>
</body>
</html>
```

Practical 5

Simple Calculator using React JS

STEPS TO PERFORM

1. Install Node.js from nodejs.org and check it with `node -v` in the terminal.
2. Create the project: `npx create-react-app calculator-app`.
3. Go into the folder: `cd calculator-app`.
4. Open the project in VS Code and replace the contents of `src/App.js` and `src/App.css` with the code below.
5. Start the app: `npm start`. The browser opens `http://localhost:3000`.
6. Test: click 7, then +, then 8, then =. The display shows 15.
7. Click C to clear the display.
8. Press `Ctrl + C` in the terminal to stop the server.

PROGRAM / DETAILS

Write a program to create a simple calculator Application using React JS

Creating a simple calculator application using React JS involves using React components, managing state, and handling events like button clicks.

App.js

```
import React, { useState } from 'react';
import './App.css';
function App() {
// State to store the current value, operation, and previous value
const [currentInput, setCurrentInput] = useState('');
const [previousInput, setPreviousInput] = useState('');
const [operation, setOperation] = useState('');
// Function to handle number button clicks
const handleNumberClick = (number) => {
setCurrentInput(currentInput + number);
};
// Function to handle operation button clicks (+, -, *, /)
const handleOperationClick = (operator) => {
setOperation(operator);
setPreviousInput(currentInput);
setCurrentInput('');
};
// Function to clear the current input
const handleClear = () => {
setCurrentInput('');
setPreviousInput('');
setOperation('');
};
```

```
// Function to evaluate the result of the current operation
const handleEvaluate = () => {
if (previousInput && operation && currentInput) {
const prev = parseFloat(previousInput);
const current = parseFloat(currentInput);
let result;
// Perform the operation
switch (operation) {
case '+':
result = prev + current;
break;
case '-':
result = prev - current;
break;
case '*':
result = prev * current;
break;
case '/':
result = prev / current;
break;
default:
return;
}
// Set the result to the current input and clear operation and previous
    input
setCurrentInput(result.toString());
setPreviousInput('');
setOperation('');
}
};
// Rendering the calculator UI
return (
<div className="calculator">
<div className="display">
<div className="previous-input">
{previousInput} {operation}
</div>
<div className="current-input">{currentInput}</div>
</div>
<div className="buttons">
<button onClick={() => handleNumberClick('7')}>7</button>
<button onClick={() => handleNumberClick('8')}>8</button>
<button onClick={() => handleNumberClick('9')}>9</button>
<button onClick={() => handleOperationClick('/')}>/</button>
<button onClick={() => handleNumberClick('4')}>4</button>
<button onClick={() => handleNumberClick('5')}>5</button>
<button onClick={() => handleNumberClick('6')}>6</button>
<button onClick={() => handleOperationClick('*')}>*</button>
<button onClick={() => handleNumberClick('1')}>1</button>
<button onClick={() => handleNumberClick('2')}>2</button>
<button onClick={() => handleNumberClick('3')}>3</button>
<button onClick={() => handleOperationClick('-')}>-</button>
```

```
<button onClick={() => handleNumberClick('0')}>0</button>
<button onClick={handleClear}>C</button>
<button onClick={handleEvaluate}>=</button>
<button onClick={() => handleOperationClick('+')}>+</button>
</div>
</div>
);
}
export default App;
```

App.css:

```
.calculator {
width: 260px;
margin: 100px auto;
background-color: #f4f4f4;
padding: 20px;
border-radius: 10px;
box-shadow: 0 4px 8px rgba(0, 0, 0, 0.2);
}
.display {
text-align: right;
margin-bottom: 20px;
}
.previous-input {
font-size: 16px;
color: #888;
}
.current-input {
font-size: 32px;
font-weight: bold;
}
.buttons button {
width: 50px;
height: 50px;
margin: 5px;
font-size: 18px;
border: 1px solid #ccc;
background-color: #fff;
cursor: pointer;
border-radius: 5px;
}
.buttons button:hover {
background-color: #f0f0f0;
}
.buttons button:active {
background-color: #ddd;
}
```

Practical 6

Voting Application using React JS

STEPS TO PERFORM

1. Install Node.js and run `npx create-react-app voting-app`, then `cd voting-app`.
2. Open the project in VS Code.
3. Replace `src/App.js` with the App.js code below.
4. Create two new files in `src`: `VoteOptions.js` and `Result.js`, and paste the code below.
5. Replace `src/App.css` with the styling given below.
6. Run `npm start` and open `http://localhost:3000`.
7. Click the Option buttons. The vote counts increase and the Results section shows the percentages.
8. Take a screenshot of the output.

PROGRAM / DETAILS

Write a program to create a voting application using React JS.

Creating a voting application using React JS involves several key steps: setting up the project, creating the components, managing state, and displaying the results. Here's how you can build a simple voting application:

1. App.js: The main container component for managing state.
2. VoteOptions.js: A component to display voting options.
3. Result.js: A component to display the results of the voting.

1. App.js

Here we will manage the state of the voting options and the number of votes for each option. We will also handle the logic for voting and displaying results.

```
import React, { useState } from 'react';
import './App.css';
import VoteOptions from './VoteOptions';
import Result from './Result';
function App() {
const [votes, setVotes] = useState({
option1: 0,
option2: 0,
option3: 0,
```

```
});
const handleVote = (option) => {
setVotes({
...votes,
[option]: votes[option] + 1,
});
};
const totalVotes = votes.option1 + votes.option2 + votes.option3;
return (
<div className="App">
<h1>Vote for your favorite option</h1>
<VoteOptions votes={votes} onVote={handleVote} />
{totalVotes > 0 && <Result votes={votes} />}
</div>
);
}
export default App;
```

2. VoteOptions.js

This component will display the voting options and allow the user to vote.

```
import React from 'react';
function VoteOptions({ votes, onVote }) {
return (
<div>
<h2>Choose an Option</h2>
<button onClick={() => onVote('option1')}>Option 1</button>
<span> ({votes.option1} votes)</span>
<br />
<button onClick={() => onVote('option2')}>Option 2</button>
<span> ({votes.option2} votes)</span>
<br />
<button onClick={() => onVote('option3')}>Option 3</button>
<span> ({votes.option3} votes)</span>
<br />
</div>
);
}
export default VoteOptions;
```

3. Result.js

This component will display the results of the voting.

```
import React from 'react';
function Result({ votes }) {
const totalVotes = votes.option1 + votes.option2 + votes.option3;
const percentage = (optionVotes) => {
return totalVotes > 0 ? ((optionVotes / totalVotes) * 100).toFixed(2) :
    0;
};
return (
<div>
<h2>Results</h2>
```

```
<p>Option 1: {votes.option1} votes ({percentage(votes.option1)}%)</p>
<p>Option 2: {votes.option2} votes ({percentage(votes.option2)}%)</p>
<p>Option 3: {votes.option3} votes ({percentage(votes.option3)}%)</p>
<p>Total Votes: {totalVotes}</p>
</div>
);
}
export default Result;
```

4. Styling (optional)

To make the app look better, you can add some simple styles in App.css.

```
.App {
text-align: center;
padding: 20px;
}
button {
margin: 5px;
padding: 10px;
font-size: 16px;
}
h1 {
color: #333;
}
h2 {
color: #444;
}
```

Practical 7

Accessing MongoDB from Node.js

STEPS TO PERFORM

1. Install Node.js and MongoDB Community Server (or create a free MongoDB Atlas cluster).
2. Create a folder: `mkdir node-mongo-app`, then `cd node-mongo-app`.
3. Initialise the project: `npm init -y`.
4. Install the packages: `npm install express mongoose`.
5. Create `index.js` and paste the server code below.
6. Start MongoDB in a separate terminal with `mongod`.
7. Run the server: `node index.js`. The console shows Connected to MongoDB.
8. Test with Postman or curl: `POST /vote` with body {"option": "Option 1"}, then `GET /votes` and `GET /results`.
9. Take a screenshot of the output.

PROGRAM / DETAILS

Write a server side program for Accessing MongoDB from Node.js.

1. Create MongoDB Database

If you don't have MongoDB running locally, you can either install MongoDB or use a cloud-based solution like MongoDB

Atlas. If using MongoDB Atlas, you'll need a connection string (e.g.,

```
mongodb+srv://<username>:<password>@cluster0.mongodb.net/myDatabase).
For local MongoDB, MongoDB runs on mongodb://localhost:27017/myDatabase
    by default.
```

2. Create Server-Side Code

Let's create a basic Node.js server that interacts with MongoDB using the Mongoose library.

```
index.js (Main Entry Point)
const express = require('express');
const mongoose = require('mongoose');
const app = express();
const port = 3000;
// Middleware to parse incoming JSON requests
app.use(express.json());
// MongoDB connection string (update with your own connection string if
```

```
    using MongoDB Atlas)
const mongoURI = 'mongodb://localhost:27017/votingApp';
// Connect to MongoDB using Mongoose
mongoose.connect(mongoURI, {
useNewUrlParser: true,
useUnifiedTopology: true,
})
.then(() => console.log('Connected to MongoDB'))
.catch((err) => console.log('Error connecting to MongoDB:', err));
// Define a Schema for the Vote (Model)
const voteSchema = new mongoose.Schema({
option: String,
votes: { type: Number, default: 0 },
});
const Vote = mongoose.model('Vote', voteSchema);
// Route to get all vote options
app.get('/votes', async (req, res) => {
try {
const votes = await Vote.find();
res.status(200).json(votes);
} catch (err) {
res.status(500).json({ message: 'Error retrieving votes', error: err });
}
});
// Route to cast a vote (update the votes for a given option)
app.post('/vote', async (req, res) => {
const { option } = req.body;
try {
let vote = await Vote.findOne({ option });
if (!vote) {
// If the option doesn't exist, create a new vote option
vote = new Vote({ option, votes: 1 });
await vote.save();
} else {
// Increment the votes for the existing option
vote.votes += 1;
await vote.save();
}
res.status(200).json(vote);
} catch (err) {
res.status(500).json({ message: 'Error casting vote', error: err });
}
});
// Route to get the results of the voting
app.get('/results', async (req, res) => {
try {
const results = await Vote.find().sort({ votes: -1 }); // Sort by votes
    in descending order
res.status(200).json(results);
} catch (err) {
res.status(500).json({ message: 'Error retrieving results', error: err
    });
```

```
}
});
// Start the server
app.listen(port, () => {
console.log(`Server running on http://localhost:${port}`);
});
```

3. Create MongoDB Schema (Optional)

In the code above, we define a Mongoose schema to structure the vote options. Each vote option will have:

```
- option: The name of the voting option (e.g., "Option 1").
- votes: The number of votes for that option (default is 0).
```

This schema is used to create the Vote model that interacts with the MongoDB database.

3. Run MongoDB (for Local MongoDB)

Make sure you have MongoDB running locally:

```
mongod
```

5. Test the API

Now you can test your API using Postman or CURL:

```
- Get all vote options: GET http://localhost:3000/votes
- Cast a vote: POST http://localhost:3000/vote with a JSON body like:
{
"option": "Option 1"
}
- Get voting results: GET http://localhost:3000/results
```

6. Optional Improvements

You can further improve this application with features such as:

```
- Validation (using Joi or express-validator).
- Authentication (e.g., with JWT or sessions).
- Rate-limiting to avoid abuse.
- Storing user votes to ensure each user can vote only once.
```

9. Final Folder Structure node-mongo-app/ ■ |-- node_modules/ # Dependencies |-- package.json # Project metadata and dependencies |-- index.js # Main server file |-- package-lock.json # Locked dependencies version

Practical 8

Manipulating MongoDB from Node.js

STEPS TO PERFORM

1. Install Node.js and MongoDB, then create a folder `mongo-crud` and run `npm init -y`.
2. Install the packages: `npm install express mongoose`.
3. Create `index.js` and paste the code below.
4. Start MongoDB with `mongod` in another terminal.
5. Run the server: `node index.js`.
6. Open Postman. Create: `POST http://localhost:3000/items` with a JSON body (name, description, price).
7. Read: `GET /items` and `GET /items/<id>`. Copy an `_id` from the result.
8. Update: `PUT /items/<id>` with new values. Delete: `DELETE /items/<id>`.
9. Take screenshots of each request and its response.

PROGRAM / DETAILS

Write a server side program for Manipulating MongoDB from Node.js

To manipulate MongoDB from Node.js, we can use the mongoose library, which is an Object Data Modeling (ODM) library that provides a higher-level API for interacting with MongoDB.

```
index.js
connection and the routes for CRUD operations.
const express = require('express');
const mongoose = require('mongoose');
const app = express();
const port = 3000;
// Middleware to parse incoming JSON requests
app.use(express.json());
// MongoDB connection URI (replace with your connection string if using
    MongoDB Atlas)
const mongoURI = 'mongodb://localhost:27017/myDatabase';
// Connect to MongoDB
mongoose.connect(mongoURI, {
useNewUrlParser: true,
useUnifiedTopology: true,
})
.then(() => console.log('Connected to MongoDB'))
.catch((err) => console.error('Error connecting to MongoDB:', err));
```

```
// Define a Schema (Model) for MongoDB
const itemSchema = new mongoose.Schema({
name: String,
description: String,
price: Number,
});
const Item = mongoose.model('Item', itemSchema);
// --- CRUD Operations ---
// Create (POST) - Create a new item
app.post('/items', async (req, res) => {
try {
const { name, description, price } = req.body;
const newItem = new Item({ name, description, price });
await newItem.save();
res.status(201).json(newItem);
} catch (err) {
res.status(500).json({ message: 'Error creating item', error: err });
}
});
// Read (GET) - Get all items
app.get('/items', async (req, res) => {
try {
const items = await Item.find();
res.status(200).json(items);
} catch (err) {
res.status(500).json({ message: 'Error fetching items', error: err });
}
});
// Read (GET) - Get a single item by ID
app.get('/items/:id', async (req, res) => {
try {
const item = await Item.findById(req.params.id);
if (!item) {
return res.status(404).json({ message: 'Item not found' });
}
res.status(200).json(item);
} catch (err) {
res.status(500).json({ message: 'Error fetching item', error: err });
}
});
// Update (PUT) - Update an existing item by ID
app.put('/items/:id', async (req, res) => {
try {
const { name, description, price } = req.body;
const updatedItem = await Item.findByIdAndUpdate(
req.params.id,
{ name, description, price },
{ new: true }
);
if (!updatedItem) {
return res.status(404).json({ message: 'Item not found' });
}
```

```
res.status(200).json(updatedItem);
} catch (err) {
res.status(500).json({ message: 'Error updating item', error: err });
}
});
// Delete (DELETE) - Delete an item by ID
app.delete('/items/:id', async (req, res) => {
try {
const deletedItem = await Item.findByIdAndDelete(req.params.id);
if (!deletedItem) {
return res.status(404).json({ message: 'Item not found' });
}
res.status(200).json({ message: 'Item deleted successfully' });
} catch (err) {
res.status(500).json({ message: 'Error deleting item', error: err });
}
});
// Start the server
app.listen(port, () => {
console.log(`Server running on http://localhost:${port}`);
});
```

A new item is created and saved to MongoDB.

```
{
"name": "Item 1",
"description": "A sample item",
"price": 100
}
```

Updates an existing item by its ID with new values for name, description, and price.

```
{
"name": "Updated Item",
"description": "Updated description",
"price": 150
}
```

5. Testing with Postman or cURL POST (Create a new item)

```
POST http://localhost:3000/items
{
"name": "Laptop",
"description": "A high-end laptop",
"price": 1200
}
GET (Get all items)
GET http://localhost:3000/items
```

GET (Get a single item by ID)

```
GET http://localhost:3000/items/5f79a8b5bfb8f774ea517d47
PUT (Update an item)
PUT http://localhost:3000/items/5f79a8b5bfb8f774ea517d47
```

```
{
"name": "Gaming Laptop",
"description": "A laptop with high-end specs",
"price": 1500
}
DELETE (Delete an item)
DELETE http://localhost:3000/items/5f79a8b5bfb8f774ea517d47
```

Practical 9

Fetch API - Display Data

STEPS TO PERFORM

1. Create a folder named `fetch-api` in VS Code.
2. Create `script.js` and write the code that calls `fetch()` on the API URL, converts the response with `response.json()` and builds the table rows.
3. Create `employee.html`, link `script.js` in it and add a `<table id="employees">` as the container.
4. Optionally add a CSS file and style the table or cards (background, border, spacing).
5. Save both files.
6. Right-click `employee.html` and choose Open with Live Server.
7. The loader spins until the response arrives, then the employee data is shown in the table.
8. Take a screenshot of the output.

PROGRAM / DETAILS

Fetch data and display the contents in the form of a card

AIM: To Get data using Fetch API from an open-source endpoint and display the contents in the form of a card

Procedure:

1. Set up the HTML

• Create a container element in your HTML where the cards will be displayed. This could be a div with a specific ID or class.

2. Write the JavaScript Code

• Fetch the Data:

– Use the fetch() method to make a request to the open-source API endpoint.

– Handle the response:

· Check for successful response status (e.g., response.ok).

· Extract the data from the response using response.json().

• Create Card Elements:

– Iterate through the received data array.

– For each data item:

· Create a new card element (e.g., a div with a class of "card").

· Create HTML elements within the card to display the data (e.g., <h2> for titles, <p> for descriptions, <img> for images).

· Populate the elements with the corresponding data from the current item.

• Append Cards to the Container:

– Append each created card element to the container element you defined in the HTML.

3. Style the Cards

• Use CSS to style the appearance of the cards (e.g., background color, borders, fonts, spacing).

```
script.js
// api url
const api_url = "https://employeedetails.sriventech.ac.in/my/api/path";
// Defining async function
async function getapi(url) {
  // Storing response
  const response = await fetch(url);
  // Storing data in form of JSON
  var data = await response.json();
  console.log(data);
  if (response) { hideloader(); }
  show(data);
}
// Calling that async function
getapi(api_url);
// Function to hide the loader
function hideloader() {
  document.getElementById("loading").style.display = "none";
}
// Function to define innerHTML for HTML table
function show(data) {
  let tab =
      `<tr><th>Name</th><th>Office</th><th>Position</th><th>Salary</th><
      /tr>`;
  // Loop to access all rows
  for (let r of data.list) {
    tab += `<tr>
<td>${r.name} </td>
<td>${r.office}</td>
<td>${r.position}</td>
<td>${r.salary}</td>
</tr>`;
}
// Setting innerHTML as tab variable
```

```
document.getElementById("employees").innerHTML = tab;
}
employee.html
<!DOCTYPE html>
<html lang="en">
<head>
<script src="script.js"></script>
<meta charset="UTF-8" />
<meta name="viewport"
content="width=device-width, initial-scale=1.0" />
<title>Document</title>
</head>
<body>
<!-- Here a loader is created which
loads till response comes -->
<div class="d-flex justify-content-center">
<div class="spinner-border"
role="status" id="loading">
<span class="sr-only">Loading...</span>
</div>
</div>
<h1>Registered Employees</h1>
<!-- table for showing data -->
<table id="employees"></table>
</body>
</html>
```

Practical 10

NodeJS Static Server without Express

STEPS TO PERFORM

1. Install Node.js and create a project folder, for example `static-site`.
2. Inside it create a sub-folder `minimal-http-server` and save the server program as `minimal-http-server/index.js`.
3. In the main folder place your `index.html` and `style.css` (the server serves files from the parent folder).
4. Create `server.js` in the main folder with: `const server = require('./minimal-http-server');` and `server.init();`.
5. Run `node server.js` in the terminal. The console shows Server is running at http://127.0.0.1:3000.
6. Open `http://localhost:3000` in the browser. The HTML page loads with its CSS.
7. Open a wrong path such as `/abc.html` to see the 404 - File Not Found message.
8. Press `Ctrl + C` to stop the server.

PROGRAM / DETAILS

NodeJS server that serves static HTML and CSS files to the user without using Express.

Aim: To Create NodeJS server that serves static HTML and CSS files to the user without using Express.

`Procedure`

1.Import the server in a Node project

2.Place the following code in minimal-http-server/index.js

3.then import the server in your code

```
const server = require('./minimal-http-server');
```

4.Start the server as follows:

```
server.init();
```

5.The server will start on port 3000. Alternatively, you can specify the port as the first argument and hostname as the second argument to the init() method as shown below:

```
server.init(4000, 'myserver');
```

```
Program
/ Dependencies
const http = require('http');
const url = require('url');
const fs = require('fs');
const path = require('path');
// Container Object
const server = {};
// Base Directory - Assuming minimal-http-server will be accessed from
    its own folder.
const baseDir = path.join(__dirname, '../');
// Mime Types
const mimeTypes = {
'.html': 'text/html',
'.jgp': 'image/jpeg',
'.css': 'text/css',
'.js': 'text/javascript',
'.png': 'image/png',
'.ico': 'image/x-icon',
};
// Create a server
const httpServer = http.createServer((request, response) => {
const parsedUrl = url.parse(request.url, true);
// Remove leading and trailing / and \ from url path
let pathName = parsedUrl.pathname;
// Get the file extension
const extensionName = path.extname(pathName);
pathName = extensionName === undefined || extensionName === '' ?
    `${pathName}/index.html` : pathName;
const responseContentType = getContentType(pathName);
response.setHeader('Content-Type', responseContentType);
// Read the file and send the response
fs.readFile(`${baseDir}${pathName}`, (error, data) => {
if (!error) {
response.writeHead(200);
response.end(data);
} else {
console.log(error);
response.writeHead(404);
response.end('404 - File Not Found');
} });});
// Get the content type for a given path
const getContentType = pathName => {
// Set the default content type
let contentType = 'application/octet-stream';
// Set the contentType based on mime type
for (var key in mimeTypes) {
if (mimeTypes.hasOwnProperty(key)) {
if (pathName.indexOf(key) > -1) {
contentType = mimeTypes[key];
} } }
return contentType;
```

```
};
// Main method to be called to start the server. 'port' defaults to
    3000 and 'host' defaults to 127.0.0.1
server.init = (port = 3000, host = '127.0.0.1') => {
httpServer.listen(port, () => {
console.log(`\x1b[32m%s\x1b[0m`, `Server is running at
    http://${host}:${port}`);
});
};
module.exports = server;
```

OUTPUT

```
node server.js
```

This will start the server, and you should see the message Server listening on port 3000 in the console.

Result

The above program was executed successfully, and the output verified

Practical 11

NodeJS Express Server - JSON / Handlebars

STEPS TO PERFORM

1. Create a folder `express-form` and run `npm init -y`.
2. Install the packages: `npm install express express-handlebars`.
3. Create a folder `views` and inside it a folder `layouts`.
4. Save `main.handlebars` in `views/layouts` and `index.handlebars` (the form) in `views`.
5. Create `server.js` in the main folder with the code below.
6. Run `node server.js`.
7. Open `http://localhost:2000`. The Student Form appears.
8. Fill in Name, Register Number and City and click Submit. The submitted data is displayed on the next page.
9. Take a screenshot of the form and of the output.

PROGRAM / DETAILS

Create a NodeJS server using Express that stores data from a form as a JSON file

Aim : Create a NodeJS server using Express that stores data from a form as a JSON file and displays it in another page. The redirect page should be prepared using Handlebars.

Procedure :

1.Create a folder called fetch API in VS Code and create a new file JavaScript file

Now, write the Code get free API and store it in Const api_url get the used id from Api using prompt method.

2. Convert into parseInt. Using asyn function get the API and point the needed data from the

API to Card. New show function get the Emp details in the table format and send it to html file.

3.In HTML file create a Card using css file and Show the data fetched from the API. To the table. Using “Id - demo" link the css file, Script file to the html and open it in Live Server.

`Server.js`

```
const express = require ('express');
const expbs = require('express-handlebars'); const app = express();
app.use(express.urlencoded());
app.engine('handlebars', expbs.engine ({defaultLayout: false,}));
    app.set('view engine',
'handlebars');
//routing
app.get('/', function(request, response, next){
    response.render('index', { layout: false });
});
app.post('/', function(request, response, next){
    response.send(request.body);});
app.listen(2000);
=>(main.handlebar)
<html>
<head>
<title>{{{ title }}} </title>
</head>
<body>
{{{ body }}}
</body>
</html>
main.html
<html>
<head>
<title> {{{ title }}} </title>
</head>
<body>
{{{ body }}}
</body>
</html>
index.html
<h1>Student Form</h1>
<form action='/' method="post">
<Table style="font-size:20px;">
<tr>
<td><label for="name">Name:</label></td>
<td><input type="text" id="fname" name="name" placeholder="Your
    name.."></td>
</tr>
<tr>
<td><label for="reg">Register Number:</label></td>
<td><input type="text" id="lname" name="lastname" placeholder="Your
    number"></td>
</tr>
<tr>
<td><label for="city">City:</label></td>
<td><input id="subject" name="subject" placeholder="Your City"
    ></input></td>
</tr>
<tr>
<td><input type="submit" value="Submit"></td>
```

```
</tr>
</Table>
</form>
```

Output:

Result : The above program was executed successfully, and the output verified.

Practical 12

NodeJS Server with MongoDB

STEPS TO PERFORM

1. Install Node.js and MongoDB, and start MongoDB with `mongod`.
2. Create a folder `customer-app`, run `npm init -y`, then `npm install mongodb`.
3. Create `create_database.js` with the first program and run `node create_database.js`. Output: Database created!
4. Create the collection: save the second program as `create_collection.js` and run it. Output: Collection created!
5. Insert data: save the third program as `insert.js` and run it. Output: 1 document inserted.
6. Update data: save the last program as `update.js` and run it. Output: 1 document updated.
7. Open `mongosh`, run `use mydb` and `db.customers.find()` to check the stored data.
8. Take a screenshot of each output.

PROGRAM / DETAILS

```
NodeJS Server creation with MongoDB
```

Aim: To Create a NodeJS server using Express that creates, insert, updates and deletes customers details and stores them in MongoDB database. The information about the user should be obtained from a HTML form.

Procedure:

1. Project Setup

Create a new Node.js project directory:

2. Create server.js file

3. Connect to MongoDB

4. Run the server create_database.js

```
var MongoClient = require('mongodb').MongoClient;
//Create a database named "mydb":
var url = "mongodb://localhost:27017/mydb";
MongoClient.connect(url, function(err, db)
{
if (err) throw err;
```

```
console.log("Database created!");
db.close();
});
Creating a Collection
var MongoClient = require('mongodb').MongoClient;
var url = "mongodb://localhost:27017/";
MongoClient.connect(url, function(err, db) {
if (err) throw err;
var dbo = db.db("mydb");
//Create a collection name "customers":
dbo.createCollection("customers", function(err, res) {
if (err) throw err;
console.log("Collection created!");
db.close();
});
});
Insert data into Collection
var MongoClient = require('mongodb').MongoClient;
var url = "mongodb://localhost:27017/";
MongoClient.connect(url, function(err, db) {
if (err) throw err;
var dbo = db.db("mydb");
var myobj = { name: "Company Inc", address: "Highway 37" };
dbo.collection("customers").insertOne(myobj, function(err, res) {
if (err) throw err;
console.log("1 document inserted");
db.close();
});});
Update document
Update the document with the address "Valley 345" to name="Mickey" and
    address="Canyon
123":
var MongoClient = require('mongodb').MongoClient;
var url = "mongodb://localhost:27017/";
MongoClient.connect(url, function(err, db) {
if (err) throw err;
var dbo = db.db("mydb");
var myquery = { address: "Valley 345" };
var newvalues = { $set: { name: "Michael", address: "Canyon 123" } };
dbo.collection("customers").updateOne(myquery, newvalues, function(err,
    res) {
if (err) throw err;
console.log("1 document updated");
db.close();
});});
```

Practical 13

NodeJS Server with MySQL

STEPS TO PERFORM

1. Install XAMPP and start the Apache and MySQL services from the control panel.
2. Open `http://localhost/phpmyadmin` and create a database (for example `company`).
3. Create a table `emp` with columns `empno`, `empname` and `age`, and insert a few rows.
4. In the `htdocs` folder create a folder `employee`. Inside it create `connection.php` that connects to the MySQL database.
5. Create `fetch.php` in the same folder and paste the code below.
6. Open `http://localhost/employee/fetch.php` in the browser.
7. The table shows all employees whose age is greater than 25.
8. Take a screenshot of the output.

PROGRAM / DETAILS

`NodeJS Server creation with MySQL`

Aim: To Create a NodeJS server that creates, reads, updates and deletes event details and stores them in a MySQL database. The information about the user should be obtained from a HTML form.

Procedure:

1. Create database in mysql

2. Write connection.php

3. Write php coding for reading data from database

```
<?php
include_once('connection.php');
$query="select * from emp where age>25";
$result=mysql_query($query);
?>
<!DOCTYPE html>
<html>
<head>
<title> Fetch Data From Database </title>
</head>
<body>
<table align="center" border="1px" style="width:600px;
```

```
    line-height:40px;">
<tr>
<th colspan="4"><h2>Employee Record</h2></th>
</tr>
<t>
<th>Empno</th>
<th> Name </th>
<th> Age </th>
</t>
<?php
while($rows=mysql_fetch_assoc($result))
{
?>
<tr>
<td><?php echo $rows['empno']; ?></td>
<td><?php echo $rows['empname']; ?></td>
<td><?php echo $rows['age']; ?></td>
</tr>
<?php
}
?>
</table>
</body>
</html>
```

Practical 14

Simple Login Form using React JS

STEPS TO PERFORM

1. Install Node.js and run `npx create-react-app login-app`, then `cd login-app`.
2. Open the project in VS Code.
3. Replace `src/App.js` with the code below.
4. Replace `src/App.css` with the styles below.
5. Run `npm start` and open `http://localhost:3000`.
6. Test 1: enter `user1` and a wrong password. The error invalid password is shown.
7. Test 2: enter an unknown username. The error invalid username is shown.
8. Test 3: enter `user1` and `pass1`. The message User is successfully logged in appears.
9. Take a screenshot of each result.

PROGRAM / DETAILS

Create a Simple Login form using React js

Now we are creating a login form which is a very important in any application or website as you know if open any website or application you will get a message to login if you click that you will be redirected to login page in this week we will be creating login page File Structure:- App.js

```
import './App.css';
import React, { useState } from "react";
import ReactDOM from "react-dom";
function App() {
// React States
const [errorMessages, setErrorMessages] = useState({});
const [isSubmitted, setIsSubmitted] = useState(false);
// User Login info
const database = [
{
username: "user1",
password: "pass1"
},
{
username: "user2",
password: "pass2"
}
```

```
];
const errors = {
uname: "invalid username",
pass: "invalid password"
};
const handleSubmit = (event) => {
//Prevent page reload
event.preventDefault();
var { uname, pass } = document.forms[0];
// Find user login info
const userData = database.find((user) => user.username === uname.value);
// Compare user info
if (userData) {
if (userData.password !== pass.value) {
// Invalid password
setErrorMessages({ name: "pass", message: errors.pass });
} else {
setIsSubmitted(true);
}
} else {
// Username not found
setErrorMessages({ name: "uname", message: errors.uname });
}
};
// Generate JSX code for error message
const renderErrorMessage = (name) =>
name === errorMessages.name && (
<div className="error">{errorMessages.message}</div>
);
// JSX code for login form
const renderForm = (
<div className="form">
<form onSubmit={handleSubmit}>
<div className="input-container">
<label>Username </label>
<input type="text" name="uname" required />
{renderErrorMessage("uname")}
</div>
<div className="input-container">
<label>Password </label>
<input type="password" name="pass" required />
{renderErrorMessage("pass")}
</div>
<div className="button-container">
<input type="submit" />
</div>
</form>
</div>
);
return (
<div className="app">
<div className="login-form">
```

```
<div className="title">Sign In</div>
{isSubmitted ? <div>User is successfully logged in</div> : renderForm}
</div>
</div>
);
}
export default App;
----------------Now Create a App.css file in same folder----------------
App.css
.app {
font-family: sans-serif;
display: flex;
align-items: center;
justify-content: center;
flex-direction: column;
gap: 20px;
height: 100vh;
font-family: Cambria, Cochin, Georgia, Times, "Times New Roman", serif;
background-color: #f8f9fd;
}
input[type="text"],
input[type="password"] {
height: 25px;
border: 1px solid rgba(0, 0, 0, 0.2);
}
input[type="submit"] {
margin-top: 10px;
cursor: pointer;
font-size: 15px;
background: #01d28e;
border: 1px solid #01d28e;
color: #fff;
padding: 10px 20px;
}
input[type="submit"]:hover {
background: #6cf0c2;
}
.button-container {
display: flex;
justify-content: center;
}
.login-form {
background-color: white;
padding: 2rem;
box-shadow: 0 4px 8px 0 rgba(0, 0, 0, 0.2), 0 6px 20px 0 rgba(0, 0, 0,
    0.19);
}
.list-container {
display: flex;
}
.error {
color: red;
```

```
font-size: 12px;
}
.title {
font-size: 25px;
margin-bottom: 20px;
}
.input-container {
display: flex;
flex-direction: column;
gap: 8px;
margin: 10px;
}
```

Practical 15

Chat Module using HTML, CSS and JavaScript

STEPS TO PERFORM

1. Create a folder `chat-module` and a file `chat.html`.
2. Paste the HTML chat containers and the CSS (inside a `<style>` tag in the head).
3. Place two avatar pictures in the folder, or change the `src` paths of the images to your own pictures.
4. Save the file.
5. Open `chat.html` in a browser or with Live Server.
6. The messages appear as chat bubbles, with the sender on the left and the reply on the right, along with the time.
7. Take a screenshot of the output.

PROGRAM / DETAILS

AIM:Write a program to build a Chat module using HTML CSS and JavaScript?

```
<div class="container">
<img src="/w3images/bandmember.jpg" alt="Avatar">
<p>Hello. How are you today?</p>
<span class="time-right">11:00</span>
</div>
<div class="container darker">
<img src="/w3images/avatar_g2.jpg" alt="Avatar" class="right">
<p>Hey! I'm fine. Thanks for asking!</p>
<span class="time-left">11:01</span>
</div>
<div class="container">
<img src="/w3images/bandmember.jpg" alt="Avatar">
<p>Sweet! So, what do you wanna do today?</p>
<span class="time-right">11:02</span>
</div>
<div class="container darker">
<img src="/w3images/avatar_g2.jpg" alt="Avatar" class="right">
<p>Nah, I dunno. Play soccer.. or learn more coding perhaps?</p>
<span class="time-left">11:05</span>
</div>
/* Chat containers */
.container {
border: 2px solid #dedede;
background-color: #f1f1f1;
border-radius: 5px;
padding: 10px;
```

```
margin: 10px 0;
}
/* Darker chat container */
.darker {
border-color: #ccc;
background-color: #ddd;
}
/* Clear floats */
.container::after {
content: "";
clear: both;
display: table;
}
/* Style images */
.container img {
float: left;
max-width: 60px;
width: 100%;
margin-right: 20px;
border-radius: 50%;
}
/* Style the right image */
.container img.right {
float: right;
margin-left: 20px;
margin-right:0;
}
/* Style time text */
.time-right {
float: right;
color: #aaa;
}
/* Style time text */
.time-left {
float: left;
color: #999;
}
```